

1. Contigüidad en memoria

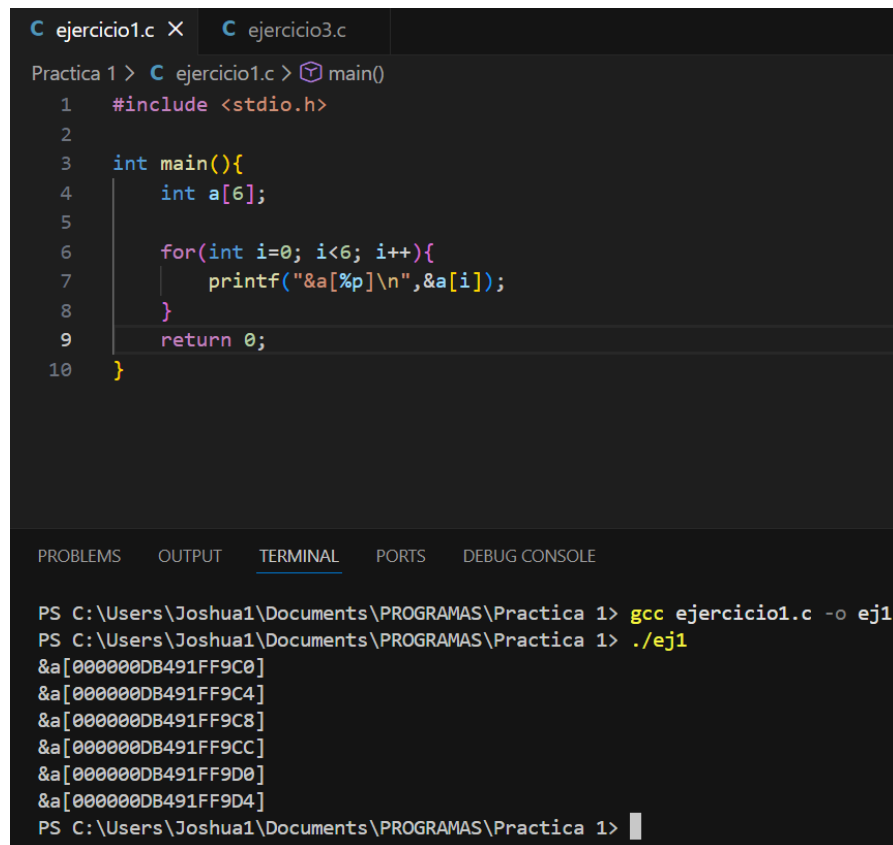
Ejercicio 1

Declara:

`int a[6];`

1. Imprime direcciones de todos los elementos.

R= Todos los elementos están en consecuencia, es decir, uno tras de otro:



```
C ejercicio1.c X C ejercicio3.c
Practica 1 > C ejercicio1.c > main()
1  #include <stdio.h>
2
3  int main(){
4      int a[6];
5
6      for(int i=0; i<6; i++){
7          printf("&a[%p]\n",&a[i]);
8      }
9      return 0;
10 }
```

PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE

```
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1> gcc ejercicio1.c -o ej1
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1> ./ej1
&a[000000DB491FF9C0]
&a[000000DB491FF9C4]
&a[000000DB491FF9C8]
&a[000000DB491FF9CC]
&a[000000DB491FF9D0]
&a[000000DB491FF9D4]
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1>
```

2. Calcula la diferencia entre direcciones consecutivas.

R= Van barrenado de 4 Bytes por segmento

3. Escribe fórmula general para dirección de `a[i]`.

R= Dirección `a[i]`=Base + `i` * tam_tipo

Ejercicio 2

Sin ejecutar determina cuál dirección es mayor: `&a[4]` o `&a[5]`

R= Por tamaño se puede decir que `a[5]` ensayo que `a[4]`

Justifica numéricamente y luego verifica ejecutando.

R= Por la fórmula se puede expresar así:

- $\&a[4] = \text{base} + 4 \times 4 = \text{base} + 16$
- $\&a[5] = \text{base} + 5 \times 4 = \text{base} + 20$

Ejercicio 3

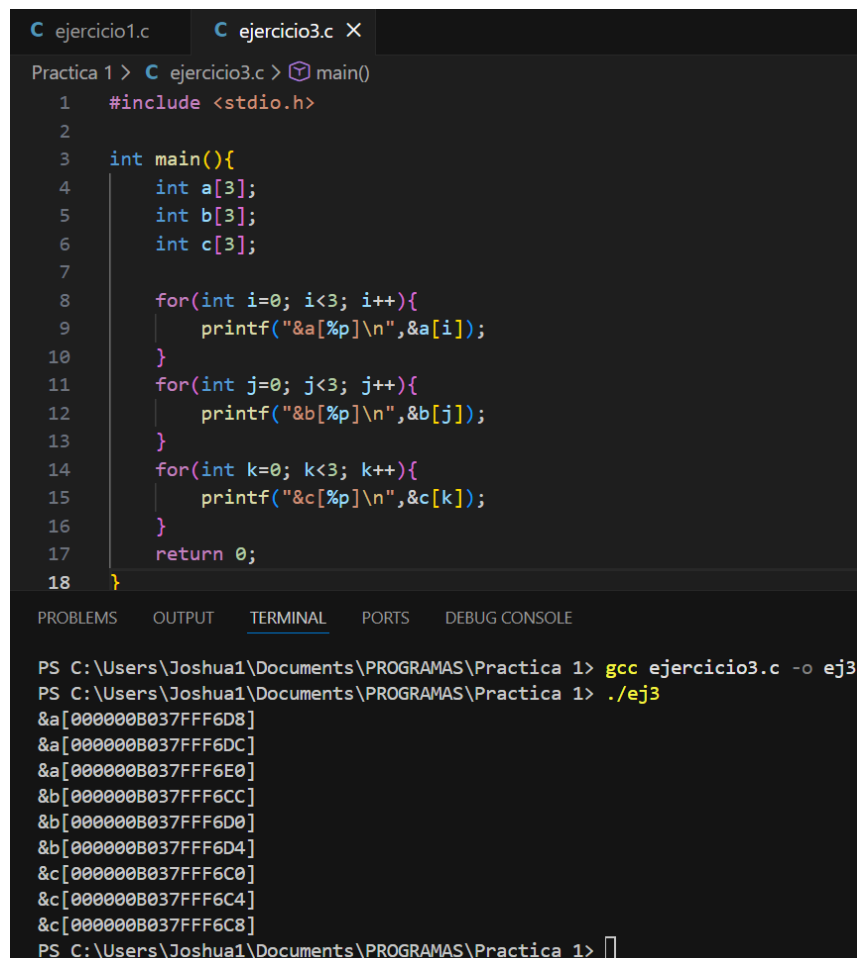
Declara:

`int a[3];`

`int b[3];`

`int c[3];`

1. Imprime todas las direcciones.



The screenshot shows a C program named `ejercicio3.c` and its execution output. The program declares three integer arrays `a`, `b`, and `c`, each of size 3. It then prints the memory address of each element in the arrays. The output shows the addresses for each element, demonstrating that the addresses increase sequentially for each array.

```
C ejercicio1.c C ejercicio3.c X
Practica 1 > C ejercicio3.c > main()
1  #include <stdio.h>
2
3  int main(){
4      int a[3];
5      int b[3];
6      int c[3];
7
8      for(int i=0; i<3; i++){
9          printf("&a[%p]\n",&a[i]);
10     }
11     for(int j=0; j<3; j++){
12         printf("&b[%p]\n",&b[j]);
13     }
14     for(int k=0; k<3; k++){
15         printf("&c[%p]\n",&c[k]);
16     }
17     return 0;
18 }
```

PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE

```
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1> gcc ejercicio3.c -o ej3
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1> ./ej3
&a[000000B037FFF6D8]
&a[000000B037FFF6DC]
&a[000000B037FFF6E0]
&b[000000B037FFF6CC]
&b[000000B037FFF6D0]
&b[000000B037FFF6D4]
&c[000000B037FFF6C0]
&c[000000B037FFF6C4]
&c[000000B037FFF6C8]
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1> 
```

2. ¿Están juntos en memoria?

R= No siempre

3. ¿De qué depende?

R= Compilador

Sistema operativo

Organización del stack

2. Cálculo de direcciones

Ejercicio 4

Salida:

$\&a[0] = 5000$

$\&a[3] = 5012$

Calcula:

tamaño del tipo

R= Para esto podemos ver que hay una diferencia de 12 unidades entre $\&a[0] = 5000$ y $\&a[3] = 5012$, por lo tanto se puede decir que hay una diferencia de 4 Bytes, que es el tamaño de cada elemento

dirección de $a[5]$

R= Su dirección sería $\&a[5] = 5020$

fórmula usada

R= Dirección = Base + i * tam tipo

Ejercicio 5

Un estudiante afirma:

Si $a[0] = 1000$ entonces $a[10] = 1010$.

Demuestra formalmente si es correcto.

R= Esto es incorrecto y se puede saber por la fórmula de Dirección = Base + i * tam_tipo donde $a[10] = 1000 + 10 \times 4 = 1040$

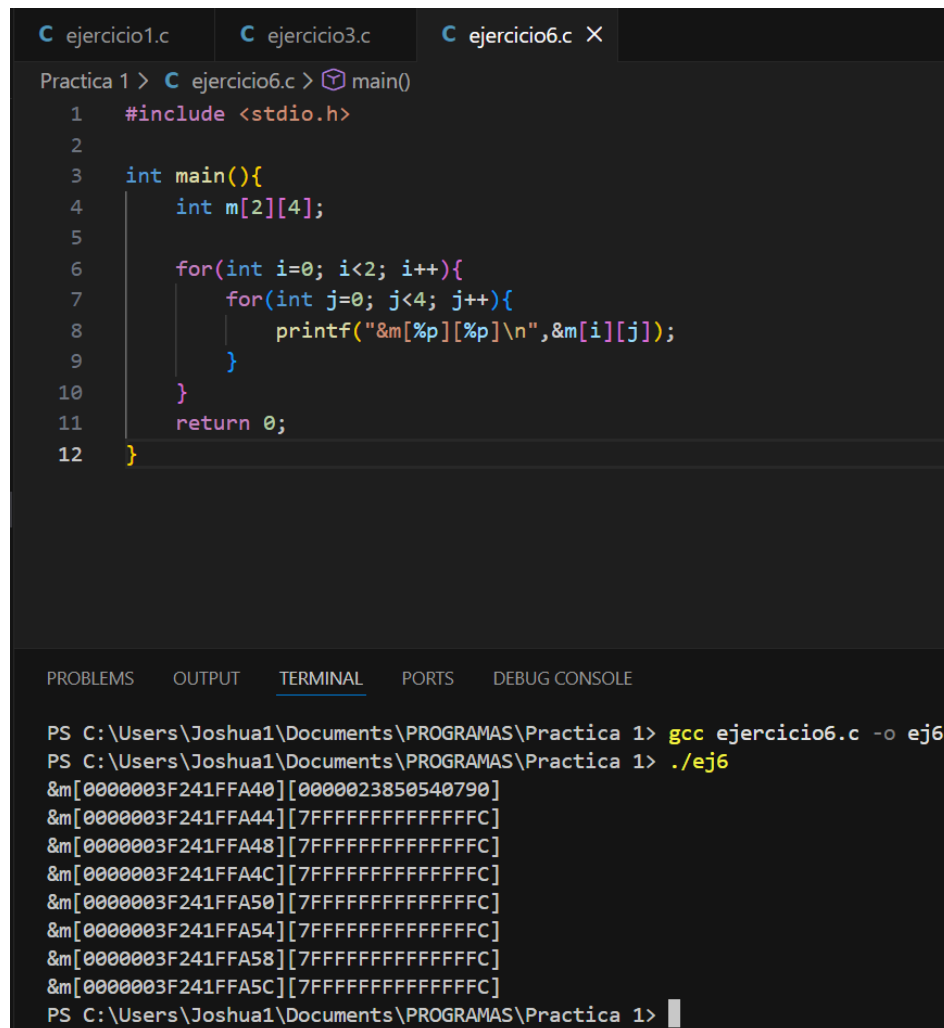
3. Arreglos bidimensionales

Ejercicio 6

Declara:

`int m[2][4];`

1. Imprime direcciones de todos los elementos.



The screenshot shows a C program in a code editor and its execution in a terminal. The code defines a 2x4 integer array `m` and prints the memory address of each element. The terminal output shows the addresses for each of the 8 elements in row-major order.

```
ejercicio1.c  ejercicio3.c  ejercicio6.c X
Practica 1 > C ejercicio6.c > main()
1  #include <stdio.h>
2
3  int main(){
4      int m[2][4];
5
6      for(int i=0; i<2; i++){
7          for(int j=0; j<4; j++){
8              printf("&m[%p][%p]\n", &m[i][j]);
9          }
10     }
11     return 0;
12 }
```

```
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1> gcc ejercicio6.c -o ej6
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1> ./ej6
&m[0000003F241FFA40][0000023850540790]
&m[0000003F241FFA44][7FFFFFFFFFFFFFFF]
&m[0000003F241FFA48][7FFFFFFFFFFFFFFF]
&m[0000003F241FFA4C][7FFFFFFFFFFFFFFF]
&m[0000003F241FFA50][7FFFFFFFFFFFFFFF]
&m[0000003F241FFA54][7FFFFFFFFFFFFFFF]
&m[0000003F241FFA58][7FFFFFFFFFFFFFFF]
&m[0000003F241FFA5C][7FFFFFFFFFFFFFFF]
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1>
```

2. Determina si almacenamiento es por filas o columnas.

R= Por filas

3. Deduce la fórmula de dirección de `m[i][j]`.

R= Dirección = Base + ((i * columnas) + j) * tam

Ejercicio 7

Sin ejecutar determina cuál dirección es mayor:

`&m[0][3]`

`&m[1][0]`

Justifica matemáticamente.

R= Las columnas son 4, esto lo usamos para lo siguiente:

- $\&m[0][3] = \text{base} + (0 \times 4 + 3) \times 4 = \text{base} + 12$
- $\&m[1][0] = \text{base} + (1 \times 4 + 0) \times 4 = \text{base} + 16$

Por lo que podemos decir que es mas grande la direccion de `&m[1][0]`

Ejercicio 8

Datos:

`base = 2000`

`sizeof(int)=4`

`m[1][2] = 2024`

Calcula:

número de columnas

$R = 2024 = 2000 + (1 \cdot C + 2) \cdot 4$

$24 = (C + 2) \cdot 4$

$6 = C + 2$

$C = 4 \rightarrow \text{Columnas}$

tamaño total

R= Para esto sabemos que son 2 filas, entonces: $2 \times 4 \times 4 = 32$ bytes

4. Límites del arreglo

Ejercicio 9

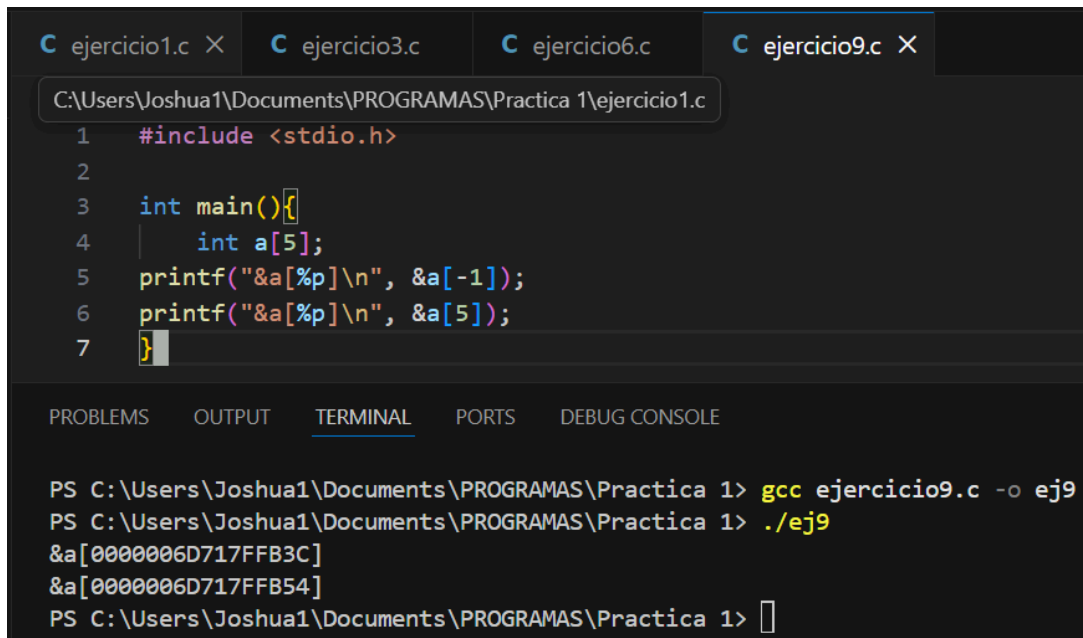
Declara:

`int a[5];`

Imprime:

`&a[-1]`

`&a[5]`



The screenshot shows a C program in a code editor with tabs for 'ejercicio1.c', 'ejercicio3.c', 'ejercicio6.c', and 'ejercicio9.c'. The active file is 'ejercicio1.c', which contains the following code:

```
1  #include <stdio.h>
2
3  int main(){
4      int a[5];
5      printf("&a[%p]\n", &a[-1]);
6      printf("&a[%p]\n", &a[5]);
7  }
```

Below the code editor is a terminal window with tabs for 'PROBLEMS', 'OUTPUT', 'TERMINAL', 'PORTS', and 'DEBUG CONSOLE'. The 'TERMINAL' tab is active, showing the following commands and output:

```
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1> gcc ejercicio9.c -o ej9
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1> ./ej9
&a[0000006D717FFB3C]
&a[0000006D717FFB54]
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1> 
```

Explica:

por qué compila

R= C no compila límites

por qué es peligroso

R= Accede a memoria ajena

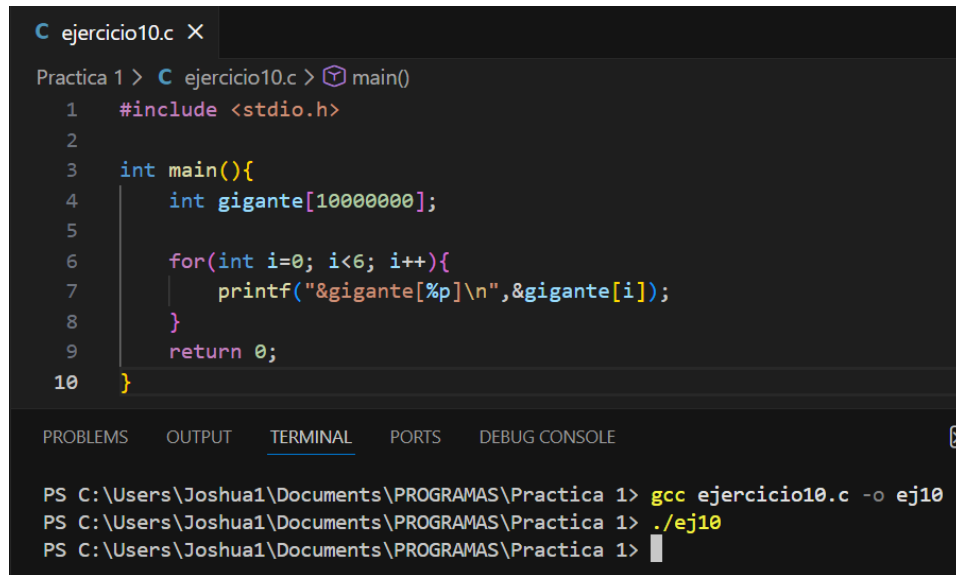
qué revela sobre memoria

R= La memoria es continua

Ejercicio 10

Declara:

`int gigante[10000000];`



```
C ejercicio10.c X
Practica 1 > C ejercicio10.c > main()
1  #include <stdio.h>
2
3  int main(){
4      int gigante[10000000];
5
6      for(int i=0; i<6; i++){
7          printf("&gigante[%p]\n",&gigante[i]);
8      }
9      return 0;
10 }
```

PROBLEMS OUTPUT **TERMINAL** PORTS DEBUG CONSOLE

```
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1> gcc ejercicio10.c -o ej10
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1> ./ej10
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1>
```

Como se puede ver como tal no es que haya fallado si no que al compilarlo no aparece el resultado, por lo que no sabría si es un fallo

Si falla:

1. explica causa
2. indica dónde se almacena
3. propone solución sin memoria dinámica

5. Nivel reto

Ejercicio 11

Salida:

`&a[0] = 1000`

`&a[1] = 1008`

`&a[2] = 1016`

Determina:

tamaño del tipo

R= Podemos ver que hay una diferencia de 8 unidades por lo que podemos decir que el tamaño de director es de 8 Bytes

tipo posible

R= Puede ser de tipo Double

fórmula usada

R= base + j×8

Ejercicio 12

Se sabe:

int m[3][5];

base = 4000

Calcula:

1. dirección de m[2][4]

R= 4000+(2×5+4)×4 → 4000+14×4=4056

2. dirección de m[1][3]

R= 4000+(1×5+3)×4 → 4000+8×4=4032

3. diferencia entre ambas

R= 4056-4032 = 24 bytes

Ejercicio 13

Elaborar un programa que al recibir como dato una matriz cuadrada, asigne valores a un arreglo unidimensional aplicando el siguiente criterio

$$B[i] = \begin{cases} \sum_{j=0}^i A[i][j] & \text{si } (i \% 3) = 1 \\ \prod_{j=1}^{n-1} A[j][i] & \text{si } (i \% 3) = 2 \\ (\prod_{j=0}^{n-1} A[j][i]) / (\sum_{j=0}^i A[j][i]) & \text{de otra forma} \end{cases}$$

```
#include <stdio.h>
```

```
int main() {  
    int n;
```

```
    printf("Ingrese el tamaño de la matriz: ");  
    scanf("%d", &n);
```

```
    float A[n][n];  
    float B[n];
```

```
    // Lectura de la matriz  
    printf("\nIngrese los elementos de la matriz:\n");  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            scanf("%f", &A[i][j]);  
        }  
    }  
}
```

```
    // Cálculo del arreglo B  
    for (int i = 0; i < n; i++) {
```

```
        // Caso i % 3 == 1
```



```

if (i % 3 == 1) {

    float suma = 0;

    for (int j = 0; j <= i; j++) {
        suma += A[i][j];
    }

    B[i] = suma;
}

// Caso i % 3 == 2
else if (i % 3 == 2) {

    float producto = 1;

    for (int j = 1; j < n; j++) {
        producto *= A[j][i];
    }

    B[i] = producto;
}

// Otro caso (i % 3 == 0)
else {

    float producto = 1;
    float suma = 0;

    for (int j = 0; j < n; j++) {
        producto *= A[j][i];
    }

    for (int j = 0; j <= i; j++) {
        suma += A[j][i];
    }

    // Evitar división entre cero
    if (suma != 0) {
        B[i] = producto / suma;
    } else {
        B[i] = 0;
        printf("Advertencia: division entre cero en i = %d\n", i);
    }
}
}

// Mostrar resultado

```

```

    printf("\nArreglo B:\n");
    for (int i = 0; i < n; i++) {
        printf("B[%d] = %.2f\n", i, B[i]);
    }

    return 0;
}

```

C ejercicio13.c X

Practica 1 > C ejercicio13.c > main()

```

1  #include <stdio.h>
2
3  int main() {
4      int n;
5
6      printf("Ingrese el tamaño de la matriz: \n");
7      scanf("%d", &n);
8
9      float A[n][n];
10     float B[n];
11
12     // Lectura de la matriz
13     printf("\nIngrese los elementos de la matriz:\n");
14     for (int i = 0; i < n; i++) {

```

PROBLEMS

OUTPUT

TERMINAL

PORTS

DEBUG CONSOLE

```

PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1> gcc ejercicio13.c -o ej13
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1> ./ej13
Ingrese el tamaño de la matriz:
2

Ingrese los elementos de la matriz:
12
23
34
45

Arreglo B:
B[0] = 34.00
B[1] = 79.00
PS C:\Users\Joshua1\Documents\PROGRAMAS\Practica 1>

```